

计算机图形学

课程作业 2：网格编辑

[方林升-102300311]

一、概述

任务 1 (贝塞尔曲线)：实现基于 1D de Casteljau 的贝塞尔曲线生成：读入 .bzc 控制点序列，使用参数 $t \in [0,1]$ 对相邻控制点做线性插值 $p'_i = (1 - t)p_i + t p_{i+1}$ ，递归地在每一层产生新的中间控制点直至收敛为曲线上一点；把递归步骤写在 BezierCurve::evaluateStep(...) 中，并用 GUI 的 E 键逐层可视化、C 键切换完整曲线以验证正确性。

任务 2 (贝塞尔曲面)：将曲线算法扩展到曲面：对 BezierPatch 中 $n \times n$ 控制点网格，先沿行方向按参数 u 对每一行做一次 1D de Casteljau 得到一列中间点，再把这些点作为新的控制点按参数 v 再做一次 1D de Casteljau，最终得到曲面上 (u,v) 处的点；实现放在 BezierPatch::evaluateStep(...), evaluate1D(...) 与 evaluate(...)，可用 teapot.bez 在 meshedit 中检查效果。

任务 3 (加权面积法顶点法线)：在半边数据结构上为每个顶点遍历其相邻三角形，计算每个三角形的法线与面积，并以面积为权重对这些面法线做加权平均、归一化得到顶点法线，在 Vertex::normal() 中实现；加载 .dae 网格后按 Q 键切换到平滑着色对比验证，注意法线应指向外侧以避免光照全黑的问题。

任务 4 (边翻转 Edge Flip)：在 HalfedgeMesh::flipEdge(...) 中对选中的内部边执行局部重网格化：将共享该边的两三角形改连为另一条对角边，严格保持半边、顶点、边、面的所有指针一致性（可利用提供的检查/调试思路）；边界边不得翻转，操作应为 $O(1)$ 且不新增/删除元素；在 GUI 中选边按 F 测试，多次翻转以验证鲁棒性。

任务 5 (边分割 Edge Split) : 在 HalfedgeMesh::splitEdge(...) 中把选中边按其中点插入新顶点 m , 把两相邻三角形改造成四个小三角形; 该操作会新建顶点、边和面, 需完整更新所有相互引用的半边指针关系, 返回新插入的 VertexIter; 基础要求可先忽略分割边界边 (遇到边界直接返回), 在 GUI 中选边按 S 进行验证, 并与翻转交替测试正确性。

任务 6 (Loop Subdivision 上采样) : 在 MeshResampler::upsample(...) 中实现一次 Loop 细分包含两步: ① 拆分每条“原始边”, 并仅翻转“旧顶点-新顶点”连成的蓝色边, 从而把每个三角形做 4-1 细分; ② 依据 Loop 权重更新顶点位置——新顶点取 $\frac{3}{8}(A+B) + \frac{1}{8}(C+D)$, 旧顶点按 $P + u \sum \text{邻居}$ (n 为度, u 随度定义) 更新; 可多次执行得到逐步更平滑的网格, 并用文档中的权重示意图对照检查。

二、贝塞尔曲线和曲面

1. 基于 1D de Casteljau 绘制贝塞尔曲线

给定控制点 $p_0, \dots, p_n$ 与参数 $t \in [0,1]$ de Casteljau 算法通过逐层线性插值得到曲线上点:

➤ 第一层: $p_i^{(1)}(t) = (1-t)p_i + tp_{i+1}, i = 0, \dots, n-1$

➤ 第二层: $p_i^{(2)}(t) = (1-t)p_i^{(1)} + tp_{i+1}^{(1)}$

➤ 这样迭代直到只剩一个点 $p_0^{(n)}(t)$, 它就是贝塞尔曲线在参数 t 的位置

这个过程只用到仿射组合 (线性插值), 数值稳定、对任意阶都适用, 单次求值时间复杂度 $O(n^2)$ 。

在任务 1 中, 我在 evaluateStep 函数中实现了“逐层插值”的动作:

```

1 std::vector<Vector2D> BezierCurve::evaluateStep(std::vector<Vector2D> const &points)
2 {
3     if (points.size() <= 1) return points;
4
5     std::vector<Vector2D> nxt;
6     nxt.reserve(points.size() - 1);
7
8     for (size_t i = 0; i + 1 < points.size(); ++i) {
9         Vector2D pi = points[i];
10        Vector2D pj = points[i + 1];
11        Vector2D pij = (1.0f - t) * pi + t * pj;
12        nxt.push_back(pij);
13    }
14
15    return nxt;
16 }

```

图 1

在求曲线上一个点时（例如 `evaluatePoint(t)`）：从原始控制点开始 `lvl = controlPoints`，不断调用 `lvl = evaluateStep(lvl)`，直到 `lvl.size() == 1`，返回 `lvl[0]` 作为该 `t` 的曲线点。

在画整条曲线时，我对 `t` 在 `[0,1]` 进行采样。采样越密，曲线越光滑。

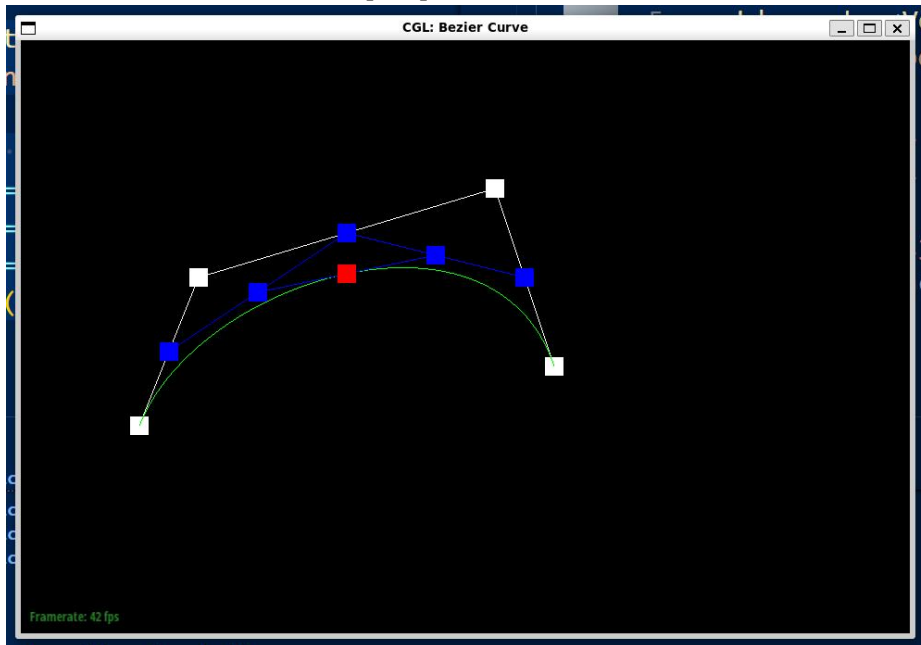


图 2：绘制 `curve1.bzc`

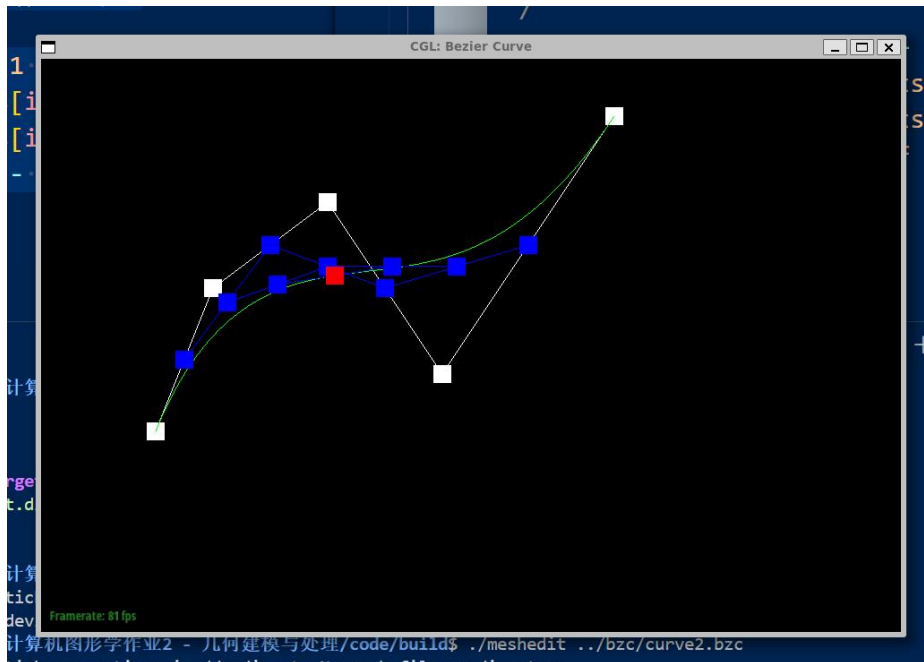


图 3：绘制 curve2.bzc

2. 基于 1D de Casteljau 绘制贝塞尔曲面

在贝塞尔曲线中，de Casteljau 算法通过对一条控制折线做逐层线性插值，最终在参数 $t \in [0,1]$ 处得到唯一的点。将这个思想推广到曲面时，我们不再只有一条控制折线，而是拥有一个 $m \times n$ 的控制点网格 $\{P_{ij}\}$ 。曲面的参数域变成了 $(u,v) \in [0,1]^2$ 。直观地说，二维扩展就是“先沿一条轴把网格压扁成一条折线，再沿另一条轴把这条折线压成一个点”。整个过程依旧只使用加法和乘法，保持了曲线版本的数值稳定性和凸包性质。

更具体地讲，可以固定列索引 j ，对每一列 $\{P_{0j}, P_{1j}, \dots, P_{(m-1)j}\}$ 按参数 u 执行一次一维 de Casteljau，得到一组中间点 $Q_j(u)$ 。这些 $Q_j(u)$ 构成了一条沿列方向的“中间折线”。接着，把这条折线当作一维控制折线，再以参数 v 执行第二次 de Casteljau，得到 $S(u,v)$ 。先做 u 再做 v ，或先做 v 再做 u ，结果完全相同，因为线性插值是可交换的并且整个过程是仿射不变的。这种“分两次做一维”的思路正是二维 de Casteljau 的核心。

这种扩展还有一个几何上的好处：曲面始终位于控制网格的凸包之内。因为每一层都是在做凸组合（ $(1-t)$ 与 t 的线性权重），无论参数如何变化，都不会“跑”出控制网的包络。对于渲染与碰撞检测，这个性质提供了天然的包围界，便于做裁剪、保守可见性判断和误差估计。

实现上可以分为三个层级：一步插值 `evaluateStep`、完整一维评估 `evaluate1D`、以及曲面评估 `evaluate`。`evaluateStep` 的职责是把输入点列 $(P_0, \dots, P_k)$ 变成下一层点列 $(P'_0, \dots, P'_{k-1})$ ，其中 $P'_i = (1-t)P_i + tP_{i+1}$ 。这一步只涉及相邻点的线性插值，时间复杂度为 $O(k)$ ，而且实现非常直接：遍历相邻点对，将结果写入新的向量即可。

`evaluate1D` 在 `evaluateStep` 的基础上迭代或递归调用，直到点列长度减少到 1 个为止。无论是迭代还是递归，返回的那个点就是在参数 t 处的贝塞尔曲线点。

曲面评估 `evaluate(u,v)` 则把一维评估用两次：先按行或按列对控制网做一次 `evaluate1D` 得到一条中间折线，再对这条折线做第二次 `evaluate1D`。

```

1  std::vector<Vector3D> BezierPatch::evaluateStep(std::vector<Vector3D> const &points, double t) const
2  {
3      if (points.size() <= 1) return points;
4
5      std::vector<Vector3D> nxt;
6      nxt.reserve(points.size() - 1);
7
8      double s = 1.0 - t;
9      for (size_t i = 0; i + 1 < points.size(); ++i) {
10         nxt.push_back(points[i] * s + points[i + 1] * t);
11     }
12     return nxt;
13 }
14
15 Vector3D BezierPatch::evaluate1D(std::vector<Vector3D> const &points, double t) const
16 {
17     if (points.empty()) return Vector3D();
18     std::vector<Vector3D> level = points;
19     while (level.size() > 1) {
20         level = evaluateStep(level, t);
21     }
22     return level[0];
23 }
24
25 Vector3D BezierPatch::evaluate(double u, double v) const
26 {
27     if (controlPoints.empty()) return Vector3D();
28     std::vector<Vector3D> rowEvaluated;
29     rowEvaluated.reserve(controlPoints.size());
30     for (const auto& row : controlPoints) {
31         rowEvaluated.push_back(evaluate1D(row, u));
32     }
33     return evaluate1D(rowEvaluated, v);
34 }

```

图 4：这一过程的代码实现

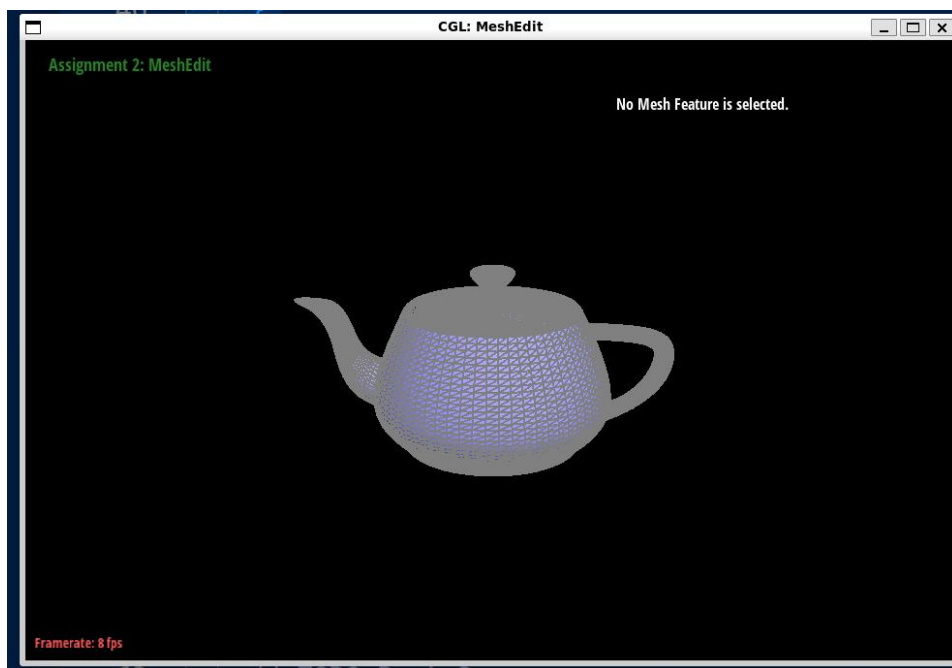


图 5: teapot.bez 渲染后的结果

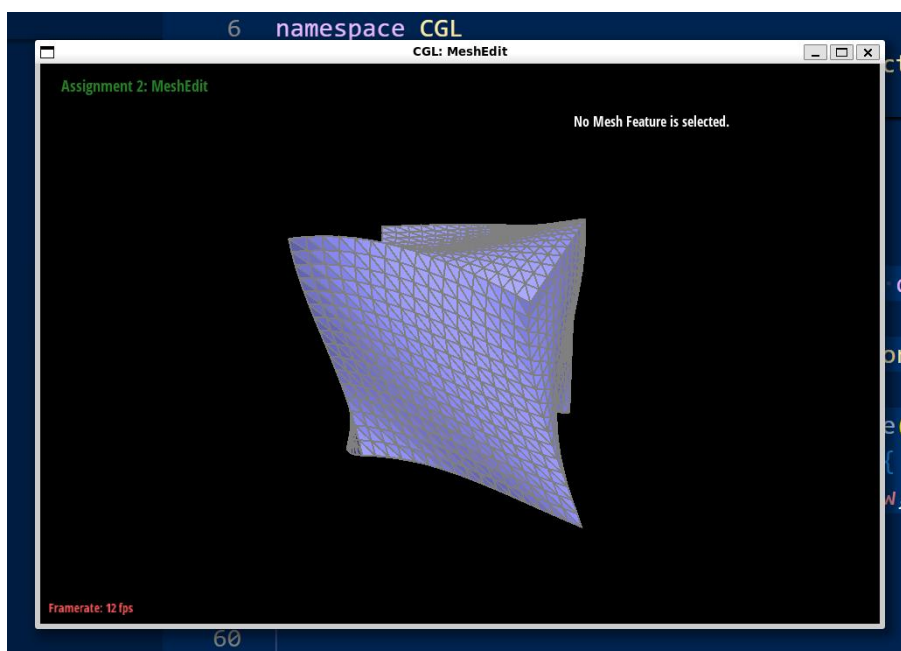


图 6: wavy_cube.bez 渲染后的结果

三、三角网格和半边数据结构

3. 基于面积加权的顶点法线计算

我将顶点法线定义为“围绕该顶点的一圈相邻三角形面法线的面积加权平均”。直观地说，哪个面在几何上占据得更“大”，它对该顶点法线的贡献就更大，从而避免窄长小面的噪声主导方向。

实现时，我从当前顶点取一个半边作为起点，按半边结构的常见环绕方式 $h \rightarrow h.twin \rightarrow h.next$ 在该顶点的一环邻域中依次访问所有相邻三角形。遇到边界面（没有真实三角形的“虚面”）就跳过，只对真实三角形进行累加。这样可以稳定地遍历一圈，既不会遗漏，也不会重复。

对每个相邻三角形，我以当前顶点为公共顶点，取从该点出发的两条边向量 e_1 和 e_2 ，计算它们的叉积 $e_1 \times e_2$ 。这个叉积向量方向就是该面的法线方向，长度等于该面面积的两倍，因此天然带有“面积权重”。把所有三角形的这类“面积向量”求和得到一个合法线向量，最后对它做一次归一化；若合向量长度接近 0（例如几何退化），就返回零向量或跳过归一化以避免数值不稳定。

采用这样得到的顶点法线后，渲染阶段对顶点法线做插值，片元使用插值后的单位法线参与光照计算，物体表面会呈现连续、平滑的明暗过渡。与不使用顶点法线（或仅用面法线做“平面着色”）相比，面积加权的顶点法线能显著减少“分面感”，在茶壶等曲面模型上能直观看到更自然的高光和阴影分布。

```

1 Vector3D Vertex::normal( void ) const
2 {
3     Vector3D n(0, 0, 0);
4
5     HalfedgeCIter h = halfedge();
6     HalfedgeCIter h0 = h;
7
8     do {
9         if (!h->face()->isBoundary()) {
10             const Vector3D& p0 = position;
11             const Vector3D& p1 = h->next()->vertex()->position;
12             const Vector3D& p2 = h->next()->next()->vertex()->position;
13
14             n += cross(p1 - p0, p2 - p0);
15         }
16         h = h->twin()->next();
17     } while (h != h0);
18
19     double len = n.norm();
20     if (len > 0.0) n /= len;
21     return n;
22 }

```

图 7：任务 3 的实现代码



图 8：两种茶壶的着色对比

4. 边翻转(Edge Flip)

我把“边翻转”实现为一个纯指针重连的局部操作。首先做合法性检查：取到目标边的两条半边 h_0 与其对偶 h_1 ，对应的两个相邻三角面 f_0 、 f_1 。若任一面为边界面则不翻转，直接返回。随后按照题图把这两个三角形围成的 6 条半

边依次命名为 $h0..h5$ ，并记四个顶点为 $a、b、c、d$ （原来中央边为 $a-b$ ，其余四条边分别为 $b-c、c-a、a-d、d-b$ ）。

接着把中央边改为连接对角顶点：将 $h0 \rightarrow \text{vertex}$ 设为 c 、 $h1 \rightarrow \text{vertex}$ 设为 d ，等价于把中央半边方向从 $a \rightarrow b / b \rightarrow a$ 变为 $c \rightarrow d / d \rightarrow c$ 。然后重连两侧三角形的环：让面 $f0$ 的三条半边依次为 $h3(c \rightarrow a) \rightarrow h4(a \rightarrow d) \rightarrow h1(d \rightarrow c)$ ，面 $f1$ 的三条半边依次为 $h5(d \rightarrow b) \rightarrow h2(b \rightarrow c) \rightarrow h0(c \rightarrow d)$ ，分别更新它们的 next 与 face 指针，并把 $f0、f1$ 的代表半边指向各自环上的任意一条。

随后更新四个顶点的代表半边，让它们各自指向一条从该顶点出发的新半边（如 $a \rightarrow d、b \rightarrow c、c \rightarrow a、d \rightarrow b$ ），并把五条相关边（中央边与四条外边）的代表半边指回到当前使用的那条。整个过程不创建/删除任何网格元素， twin/edge 关系保持不变，仅修改 $\text{vertex}/\text{next}/\text{face}$ 与若干“代表 halfedge”指针。

最后返回这条被翻转后的边迭代器。该实现是 $O(1)$ 的局部更新，保持网格的流形性与一致性；若命中边界或非三角形情况则安全早退，不做修改。

```

1  EdgeIter HalfedgeMesh::flipEdge( EdgeIter e0 )
2  {
3      HalfedgeIter h0 = e0->halfedge();
4      HalfedgeIter h1 = h0->twin();
5      FaceIter     f0 = h0->face();
6      FaceIter     f1 = h1->face();
7      if (f0->isBoundary() || f1->isBoundary()) return e0;
8
9      HalfedgeIter h2 = h0->next();
10     HalfedgeIter h3 = h2->next();
11     HalfedgeIter h4 = h1->next();
12     HalfedgeIter h5 = h4->next();
13
14     VertexIter a = h0->vertex();
15     VertexIter b = h1->vertex();
16     VertexIter c = h3->vertex();
17     VertexIter d = h5->vertex();
18
19     EdgeIter e1 = h2->edge();
20     EdgeIter e2 = h3->edge();
21     EdgeIter e3 = h4->edge();
22     EdgeIter e4 = h5->edge();
23
24     h0->vertex() = c;
25     h1->vertex() = d;
26
27     h3->next() = h4;
28     h4->next() = h1;
29     h1->next() = h3;
30
31     h5->next() = h2;
32     h2->next() = h0;
33     h0->next() = h5;
34
35     h3->face() = f0; h4->face() = f0; h1->face() = f0; f0->halfedge() = h1;
36     h5->face() = f1; h2->face() = f1; h0->face() = f1; f1->halfedge() = h0;
37
38     a->halfedge() = h4;
39     b->halfedge() = h2;
40     c->halfedge() = h3;
41     d->halfedge() = h5;
42
43     e0->halfedge() = h0;
44     e1->halfedge() = h2;
45     e2->halfedge() = h3;
46     e3->halfedge() = h4;
47     e4->halfedge() = h5;
48
49     return e0;
50 }

```

图 9：任务 4 的代码实现

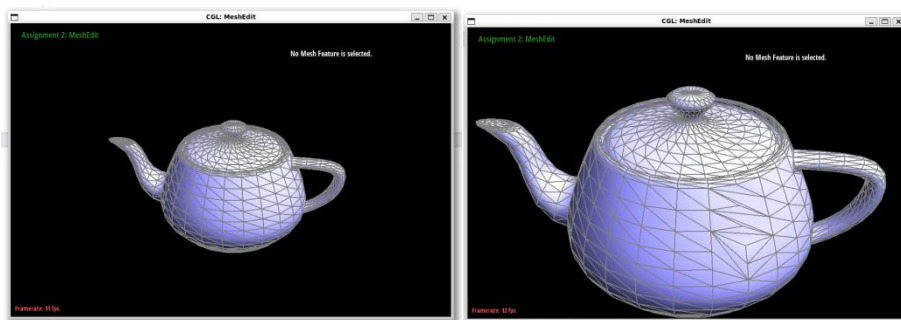


图 10：边翻转前后的茶壶图像

5. 边分割(Edge Split)

我把“边分割”实现成一个在半边数据结构上局部、可逆的拓扑更新。入口是一条边 e 。首先判断它是否为边界边，并收集它两侧（或一侧）面片的 6 条（或 3 条）相关半边与 3~4 个顶点，建立起局部的 $a-b-c-d$ （边界情形没有 d ）记号体系。随后新建一个顶点 m （位置取两端点中点），以及若干条新边/半边与必要的新面。整体策略是：复用原边的两条半边作为 $b \rightarrow m$ 、 $c \rightarrow m$ ，把其余方向用新半边补齐，再把这套半边重新编成四个（或两个）面环。

对内侧边：原来由两三角形 (a,b,c) 与 (c,b,d) 围绕 e 。分割后生成四个三角形： (b,m,a) 、 (m,c,a) 、 (c,m,d) 、 (m,b,d) 。做法是新建三条边 (m,c) 、 (a,m) 、 (d,m) 与 6 条半边；把旧的 $h(b \rightarrow c)$ 、 $h(c \rightarrow b)$ 改作 $h(b \rightarrow m)$ 、 $h(c \rightarrow m)$ ，分别与新建的 $h(m \rightarrow b)$ 、 $h(m \rightarrow c)$ 配对为 twin；再按上述四个三角的顺序设置每个面环的 next 指针与 face 指针，最后更新每条边/每个面的代表 halfedge。这样保证了半边成对、面环闭合、顶点入射环一致的三大不变量。

对边界边：原来 e 一侧是三角形 (a,b,c) ，另一侧是边界环段 $(c \rightarrow b)$ 。分割后把三角形切为 (a,b,m) 与 (a,m,c) ，同时将边界段拆成 $(c \rightarrow m)$ 与 $(m \rightarrow b)$ ，并把它接回同一个边界面的环上。实现上复用原 e 的两条半边作为 $b \rightarrow m$ 与 $c \rightarrow m$ ，新建 $(m \rightarrow b)$ 、 $(m \rightarrow c)$ 与 $(a \leftrightarrow m)$ 的半边；先重编内侧两个三角的

next/face, 再把边界的 next 改为 $(c \rightarrow m) \rightarrow (m \rightarrow b) \rightarrow \dots$, 从而维持边界环的连通与方向一致。

最后, 统一做两点收尾: (1) 把各新旧边的 $\text{edge} \rightarrow \text{halfedge}()$ 、各面的 $\text{face} \rightarrow \text{halfedge}()$ 和各顶点的 $\text{vertex} \rightarrow \text{halfedge}()$ 代表指针设为环上的任意成员; 其中新顶点 m 的代表半边特意指向“沿被切分的那条边”的方向 ($m \rightarrow b$), 满足题目要求。(2) 复制位置属性并可选择保留法线/颜色等插值。至此, 边分割既不会破坏半边结构的一致性, 又把 2 个三角稳定地改写为 4 个 (或把 1 个三角改写为 2 个) 并正确更新了边界。

```

1 Vertexiter HalfEdgeMesh::splitEdge( Edgeiter eb )
2 {
3
4     if (!eb->isBoundary())
5     {
6         Halfedgeiter hb = eb->halfedge();
7         Halfedgeiter h3 = hb->twinn();
8
9         Halfedgeiter h1 = hb->next();
10        Halfedgeiter h2 = h1->next();
11
12        Halfedgeiter h4 = h3->next();
13        Halfedgeiter h5 = h4->next();
14
15        Vertexiter vb = hb->vertex();
16        Vertexiter vc = h3->vertex();
17        Vertexiter va = h2->vertex();
18        Vertexiter vd = h5->vertex();
19
20        Facetter fl = hb->face();
21        Facetter fb = h3->face();
22
23        Vertexiter vm = newVertex();
24        vm->position = 0.5 * (vb->position + vc->position);
25
26        Edgeiter e_nc = newEdge();
27        Edgeiter e_am = newEdge();
28        Edgeiter e_dm = newEdge();
29
30        Facetter fac = newFace();
31        Facetter fbd = newFace();
32
33        Halfedgeiter hm_bm = hb;
34        Halfedgeiter hm_cn = h3;
35
36        Halfedgeiter hm_mb = newHalfedge();
37        Halfedgeiter hm_mc = newHalfedge();
38        Halfedgeiter hm_ma = newHalfedge();
39        Halfedgeiter hm_am = newHalfedge();
40        Halfedgeiter hm_md = newHalfedge();
41        Halfedgeiter hm_dm = newHalfedge();
42
43        hm_bm->twinn() = hm_mb; hm_mb->twinn() = hm_bm; hm_bm->edge() = eb; hm_mb->edge() = eb;
44        hm_cn->twinn() = hm_mc; hm_mc->twinn() = hm_cn; hm_cn->edge() = e_nc; hm_mc->edge() = e_nc;
45        hm_ma->twinn() = hm_am; hm_am->twinn() = hm_ma; hm_ma->edge() = e_am; hm_am->edge() = e_am;
46        hm_md->twinn() = hm_dm; hm_dm->twinn() = hm_md; hm_md->edge() = e_dm; hm_dm->edge() = e_dm;
47
48        hm_bm->vertex() = vb; hm_mb->vertex() = vm;
49        hm_cn->vertex() = vc; hm_mc->vertex() = vm;
50        hm_ma->vertex() = va; hm_am->vertex() = va;
51        hm_md->vertex() = vd; hm_dm->vertex() = vd;
52
53        hm_bm->next() = hm_ma; hm_ma->next() = h2; h2->next() = hm_bm;
54        hm_bm->face() = fl; hm_ma->face() = fl; h2->face() = fl; fl->halfedge() = hm_bm;
55
56        hm_mc->next() = h1; h1->next() = hm_am; hm_am->next() = hm_mc;
57        hm_mc->face() = fac; h1->face() = fac; hm_am->face() = fac; fac->halfedge() = hm_mc;
58
59        hm_cn->next() = hm_md; hm_md->next() = h5; h5->next() = hm_cn;
60        hm_cn->face() = fb; hm_md->face() = fb; h5->face() = fb; fb->halfedge() = hm_cn;
61
62        hm_mb->next() = h4; h4->next() = hm_dm; hm_dm->next() = hm_mb;
63        hm_mb->face() = fbd; h4->face() = fbd; hm_dm->face() = fbd; fbd->halfedge() = hm_mb;
64
65        eb->halfedge() = hm_bm;
66        e_nc->halfedge() = hm_cn;
67        e_am->halfedge() = hm_ma;
68        e_dm->halfedge() = hm_md;
69
70        vm->halfedge() = hm_mb;
71        return vm;
72    }
73
74    Halfedgeiter h1 = eb->halfedge();
75    if (h1->face()->isBoundary()) h2 = h1->twinn();
76    Halfedgeiter hb = h1->twinn();
77
78    Halfedgeiter h1l = h1->next();
79    Halfedgeiter h12 = h1l->next();
80
81    Vertexiter vb = h1->vertex();
82    Vertexiter vc = hb->vertex();
83    Vertexiter va = h12->vertex();
84
85    Facetter flr1 = h1->face();
86    Facetter fbd = hb->face();
87
88    Vertexiter vm = newVertex();
89    vm->position = 0.5 * (vb->position + vc->position);
90
91    Edgeiter e_nc = newEdge();
92    Edgeiter e_am = newEdge();
93
94    Facetter fac = newFace();
95
96    Halfedgeiter hm_bm = h1;
97    Halfedgeiter hm_cn = hb;
98
99    Halfedgeiter hm_mb = newHalfedge();
100    Halfedgeiter hm_mc = newHalfedge();
101    Halfedgeiter hm_ma = newHalfedge();
102    Halfedgeiter hm_am = newHalfedge();
103
104    hm_bm->twinn() = hm_mb; hm_mb->twinn() = hm_bm; hm_bm->edge() = eb; hm_mb->edge() = eb;
105    hm_cn->twinn() = hm_mc; hm_mc->twinn() = hm_cn; hm_cn->edge() = e_nc; hm_mc->edge() = e_nc;
106    hm_ma->twinn() = hm_am; hm_am->twinn() = hm_ma; hm_ma->edge() = e_am; hm_am->edge() = e_am;
107
108    hm_bm->vertex() = vb; hm_mb->vertex() = vm;
109    hm_cn->vertex() = vc; hm_mc->vertex() = vm;
110    hm_ma->vertex() = va; hm_am->vertex() = va;
111
112    hm_bm->next() = hm_ma;
113    hm_ma->next() = h12;
114    h12->next() = hm_bm;
115    hm_bm->face() = flr1; hm_ma->face() = flr1; h12->face() = flr1;
116    flr1->halfedge() = hm_bm;
117
118    hm_mc->next() = h1l;
119    h1l->next() = hm_am;
120    hm_am->next() = hm_mc;
121    hm_mc->face() = fac; h1l->face() = fac; hm_am->face() = fac;
122    fac->halfedge() = hm_mc;
123
124    Halfedgeiter hb_next = hm_cn->next();
125    hm_cn->next() = hm_mb;
126    hm_mb->next() = hb_next;
127    hm_cn->face() = fbd; hm_mb->face() = fbd;
128    fbd->halfedge() = hm_cn;
129
130    eb->halfedge() = hm_bm;
131    e_nc->halfedge() = hm_cn;
132    e_am->halfedge() = hm_ma;
133
134    vm->halfedge() = hm_mb;
135
136    return vm;
137 }
138

```

图 11：任务 5 的代码实现

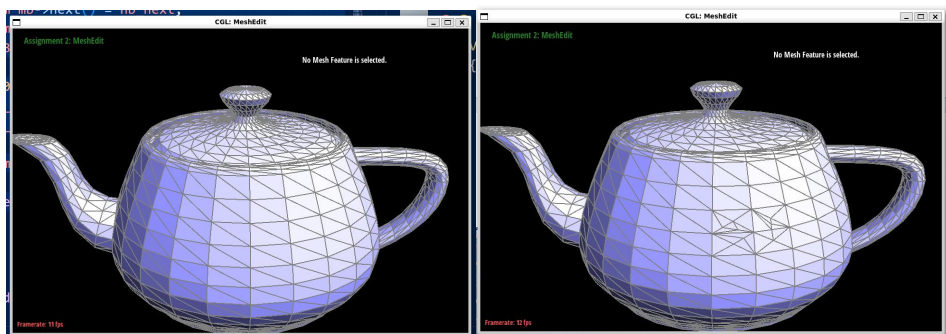


图 12：边分割前后的茶壶图像

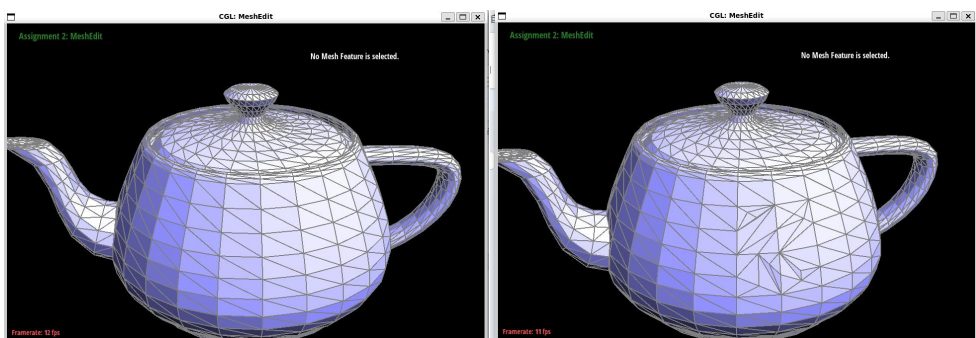


图 13：边分割翻转组合前后的茶壶图像

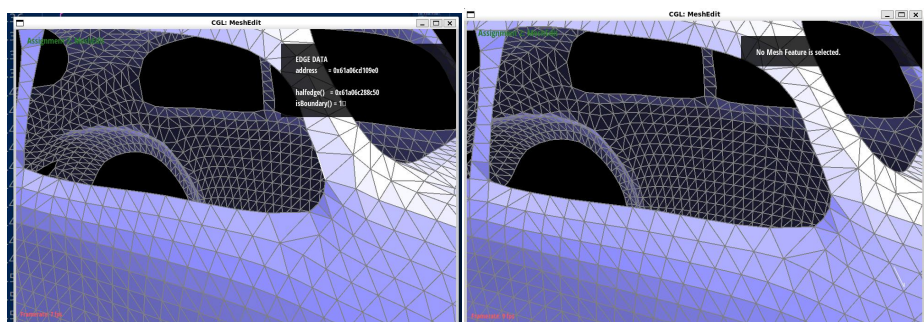


图 14：边界边分割前后的图像

6. 利用 Loop Subdivision 进行网格上采样

在实现上，我首先在不改变连通性的前提下，为所有旧顶点与旧边各自计算一个“候选位置”。对内部顶点，我按 Loop 规则用其一环邻居的加权平均来更新：有 $\mathbf{v}_{new} = (1 - nu)\mathbf{v} + u\sum \mathbf{v}_i$ 。对边界顶点，我使用边界特例 $\mathbf{v}_{new} = \frac{3}{4}\mathbf{v} +$

$\frac{1}{8}(\mathbf{v}_1 + \mathbf{v}_2)$ ，其中 $\mathbf{v}_1, \mathbf{v}_2$ 是沿边界线段的相邻点。所有这些新位置都先放到 `Vertex::newPosition`，并把旧顶点标为 `isNew=false`，以便后面区分“旧/新”元素。

接着我为每条原始边计算插值点的位置并存入 `Edge::newPosition`。对内部边 AB，取其两侧三角形对顶点 C、D，边点为 $\frac{3}{8}(A + B) + \frac{1}{8}(C + D)$ ；对边界边则取端点中点。随后我只对“起初就存在的边”执行一次 split：每次分裂生成的新顶点 m 继承该边的 `newPosition`，并被标记为 `isNew=true`。分裂后围绕 m 会出现四条边，其中与原边两个端点相连的两条边被标记为旧边 (`isNew=false`)，连接到两侧对顶点的两条被标记为新边 (`isNew=true`)，这样可以在后续翻转时精确锁定目标。

完成所有分裂后，我再进行一次“只翻转新边”的遍历：凡是 `isNew==true` 且两端分别为“一个旧顶点、一个新顶点”的内部边，执行 flip。这一步把局部连接改造成 Loop 模板所需的“新—旧”结构，使新插入的顶点与旧网格更规则地连接。最后，我把每个顶点的 `newPosition` 写回到真正的 `position`，至此一次 Loop 细分完成；此过程天然会让曲面更平滑，并带来一定体积收缩，这是该算法的固有属性。

```

1 void MeshResampler::upsample( HalfedgeMesh& mesh )
2 {
3     for (VertexIter v = mesh.verticesBegin(); v != mesh.verticesEnd(); v++) {
4         v->isNew = false;
5
6         if (v->isBoundary()) {
7             HalfedgeIter hb = v->halfedge();
8             while (!hb->isBoundary()) hb = hb->twin()->next();
9
10            HalfedgeIter hb2 = hb;
11            do { hb2 = hb2->twin()->next(); }
12            while (!hb2->isBoundary());
13
14            VertexIter v1 = hb->twin()->vertex();
15            VertexIter v2 = hb2->twin()->vertex();
16
17            v->newPosition = 0.75 * v->position + 0.125 * (v1->position + v2->position);
18        } else {
19            size_t n = v->degree();
20            double u = (n == 3) ? (3.0 / 16.0) : (3.0 / (8.0 * n));
21
22            Vector3D sum(0,0,0);
23            HalfedgeIter h = v->halfedge();
24            do {
25                sum += h->twin()->vertex()->position;
26                h = h->twin()->next();
27            } while (h != v->halfedge());
28
29            v->newPosition = (1.0 - n*u) * v->position + u * sum;
30        }
31    }
32
33    for (EdgeIter e = mesh.edgesBegin(); e != mesh.edgesEnd(); e++) {
34        e->isNew = false;
35
36        HalfedgeIter h = e->halfedge();
37        VertexIter A = h->vertex();
38        VertexIter B = h->twin()->vertex();
39
40        if (e->isBoundary()) {
41            e->newPosition = 0.5 * (A->position + B->position);
42        } else {
43            VertexIter C = h->next()->next()->vertex();
44            VertexIter D = h->twin()->next()->next()->vertex();
45            e->newPosition = (3.0/8.0) * (A->position + B->position)
46                + (1.0/8.0) * (C->position + D->position);
47        }
48    }
49
50    std::vector<EdgeIter> origEdges;
51    for (EdgeIter e = mesh.edgesBegin(); e != mesh.edgesEnd(); e++) origEdges.push_back(e);
52
53    for (EdgeIter e : origEdges) {
54        HalfedgeIter h = e->halfedge();
55        VertexIter A = h->vertex();
56        VertexIter B = h->twin()->vertex();
57        VertexIter C, D;
58        if (!e->isBoundary()) {
59            C = h->next()->next()->vertex();
60            D = h->twin()->next()->next()->vertex();
61        }
62
63        VertexIter m = mesh.splitEdge(e);
64        m->isNew = true;
65        m->newPosition = e->newPosition;
66
67        HalfedgeIter hm = m->halfedge();
68        HalfedgeIter start = hm;
69        do {
70            EdgeIter ed = hm->edge();
71            VertexIter o = hm->twin()->vertex();
72            if (o == A || o == B) ed->isNew = false;
73            else ed->isNew = true;
74            hm = hm->twin()->next();
75        } while (hm != start);
76    }
77
78    for (EdgeIter e = mesh.edgesBegin(); e != mesh.edgesEnd(); e++) {
79        if (!e->isNew || e->isBoundary()) continue;
80        VertexIter v0 = e->halfedge()->vertex();
81        VertexIter v1 = e->halfedge()->twin()->vertex();
82        if (v0->isNew ^ v1->isNew) {
83            mesh.flipEdge(e);
84        }
85    }
86
87    for (VertexIter v = mesh.verticesBegin(); v != mesh.verticesEnd(); v++) {
88        v->position = v->newPosition;
89    }
90 }
91

```

图 15: 任务 6 的实现代码

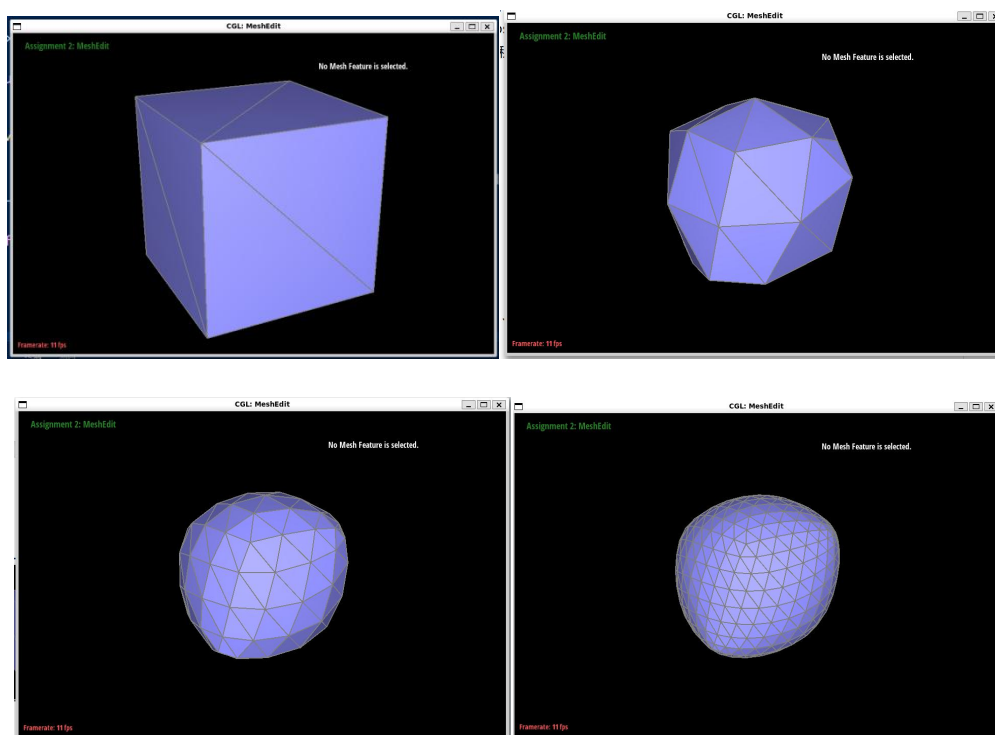


图 16：立方体进行几次迭代后的截图

从零次到多次细分的截图可以看到，立方体的八个尖角会迅速被“磨钝”，棱边也会变圆。Loop 的更新本质上是一次尺度依赖很小的平滑算子：顶点位置被邻域平均所“拉拢”，因此高曲率处（比如角和棱）被持续平抑，低曲率处变化较小。边界若未做“特征保持”，会按边界规则沿着外形向内收缩，呈现更圆滑但更小的轮廓。

继续迭代时，模型整体趋向一个近似球体，同时面片的质量也在改善：细长或锐利三角形的比例降低，越来越接近规则的六价三角网。然而，如果初始三角剖分不均匀，某些区域仍可能出现轻微的“鼓起/塌陷”差异。这并非渲染伪影，而是由连通性差异引起的真实几何偏置——Loop 的权重只看拓扑邻接和度数，不直接“看见”几何对称轴，因而拓扑上的细小不对称会被多次迭代逐步放大为可见的形变。

此外，边界与尖角的变化也能从截图中直观看到：边界点在每次迭代都被以 $\frac{3}{4}$ 与 $\frac{1}{8}$ 的比例向相邻边界点拉拢，这比内部点的权重更“强”，因此开边界的收缩感通常更明显。若在细分前沿边界先做一些均匀分割，让边界采样更密且度数分布更一致，收缩会更均匀，但在没有额外“锐边/折线保持”规则时，边界仍会被不断圆化。

立方体在导入后，每个四边形面通常被任意选择的一条对角线拆成两个三角形。对称性的问题就埋在这里：相对的两个面，很可能用的是不同方向的对角线；同一条棱的两侧，三角形的“指向”也可能不一致。Loop 的更新权重依赖顶点度与一环的三角形分布，这种对角线选择带来的拓扑非对称使得某些角点周围的度数与边走向不再镜像，长此以往，迭代会把这种极小的差异放大为可见的不对称。

要缓解这个问题，关键在于让连通性变得对称与规则。一个简单有效的做法是：在细分前，用边翻转把六个面的对角线调整为成对镜像或棋盘式交替的排布，使每个角点看到的局部三角形图案一致；同时对明显细长的三角形做翻转，尽量把网格变为更接近等边的三角形。这样做能让所有角点的度数更接近 6，Loop 模板对每个角点的作用几乎相同，因而迭代后整体形状更接近完美的对称。

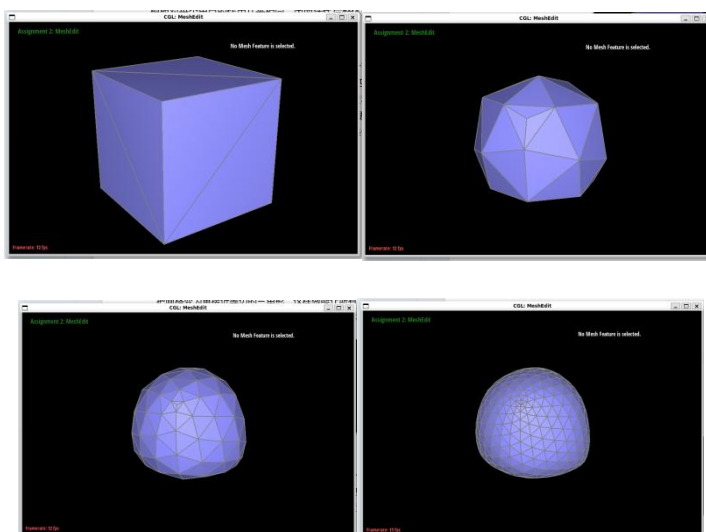


图 17：立方体预处理后进行几次迭代后的截图

进一步的预处理还可以在每个面上进行均匀分割：先沿面内对称的两条边做 split 以引入接近面心的顶点，再按需要翻转，使四个子三角形尽可能均衡。这样不仅提升了面片质量，也把连通性中潜在的偏置稀释掉。配合上述对角线对称化与“瘦长三角形翻转”，重复细分后，立方体会更加圆润且左右/前后/上下对称，同时体积收缩在各个方向上更均匀，主观观感也更自然。

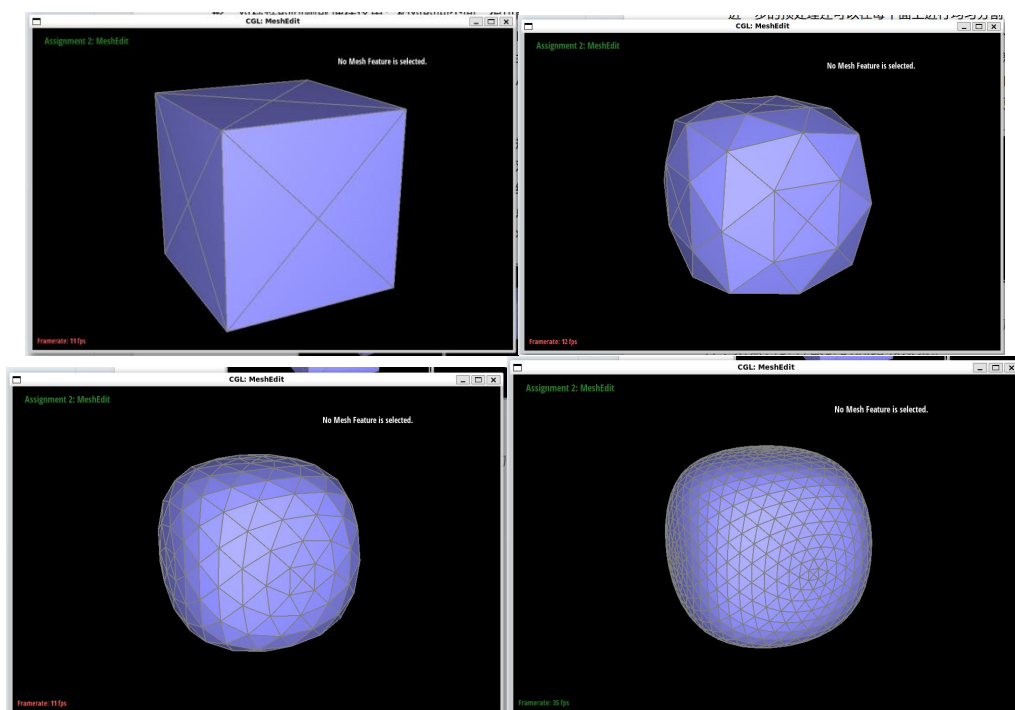


图 18：立方体进一步预处理后进行几次迭代后的截图

四、实践与思考

在整个编程与调试过程中，我主要经历了算法实现、代码结构理解、调试验证与结果分析四个阶段。初期调试重点放在熟悉 HalfedgeMesh 的拓扑关系与数据结构接口上。由于半边数据结构牵涉大量指针相互引用，任何指针更新不一致都会导致程序崩溃或模型破裂，因此在编写“边翻转”“边分割”功能时，我多次利用文档提供的 CHECK_CLOSED、CHECK_FACE、CHECK_VERTEX 等断言宏进行验证。通过这些调试工具，我逐步定位了指针错连、边界判断遗漏等问

题，并最终保证所有面环闭合、半边互为双向、顶点环绕关系正确。对于 Loop Subdivision，调试重点转向逻辑正确性与视觉效果验证。通过对不同形状模型（如茶壶、立方体）的反复上采样，我验证了算法在内点、边界点的插值权重是否合理，确保模型逐步变得平滑且保持连通性。

从实验效果上看，各个算法的优点十分明显。贝塞尔曲线与曲面的 de Casteljau 实现数值稳定、易于控制，插值过程直观且可视化效果良好。面积加权法线能显著提升模型的光滑度，使光照与阴影过渡自然。半边结构下的边翻转与分割操作都能在 $O(1)$ 时间内完成，且局部修改不影响整体拓扑。Loop Subdivision 在连续迭代后能显著提升曲面平滑度，使原始棱角分明的立方体渐变为近似球体，模型表面趋于连续可导，渲染时几乎看不到明显的分片痕迹。这些结果体现了算法在几何建模与平滑处理中的稳定性与通用性。

然而在实验中也发现了一些问题。首先，半边结构操作极易出现“指针未更新完全”或“边界特殊情况未处理”的 bug，例如在边分割时若遗漏更新某个顶点或面代表半边，会导致数据结构失稳。其次，Loop 细分算法在边界和高曲率区域存在明显的体积收缩与轻微不对称性，尤其在初始网格连通性不均匀时更为明显。此外，对于边界顶点，若未加入“锐边保持”或“特征约束”，边界线条会被逐步圆化，导致模型外形的细节丢失。

针对这些问题，我尝试了几种改进思路。对于半边操作，增加了拓扑一致性检查函数，每完成一次结构修改即验证面环与半边成对性，有效减少了调试难度。对于 Loop 上采样的形变与不对称，我通过两种方式改善：一是预处理网格拓扑，在细分前用边翻转将对角线方向统一或交替排列，使各角点的局部拓扑一致；二是引入边界特例与对称化分割，即在边界附近执行额外的边分割以保证度数分布均衡。改进后，立方体细分生成的模型更加圆润且保持全局对称，体积收缩趋势也显著减弱。整体来看，这些调试与优化过程不仅提升了算法稳定性，也加深了我对几何处理算法内部原理的理解。